

PROJETO INTEGRADOR — IoT

Diagnóstico de Informações — O que já temos

Faculdade Senac PE • CST em ADS— 4º Período
Fase Maker • Controle de Acesso IoT via QR Code
Gerado em: 07/06/2026

Equipe

Hardware: Amanda Cruz, Phellipe Leandro
Frontend: Isabel Vitória, Lucas Eloi
Backend: Nikolas Messias, Ronald Paixão, Vinicius Manoel
Documentação: Gilberto Quintino

2. Informações já disponíveis

2.1 Identificação do Projeto

Nome do projeto	Acesso Inteligente Senac
Nome da equipe	Fase Maker
Temática	Controle de acesso acadêmico integrado a aplicativo mobile e dispositivo IoT
Problema central	Informações acadêmicas dispersas; portal pouco prático no celular; ausência de dados de ocupação em tempo real para a gestão
Solução	App mobile com QR Code dinâmico + hardware IoT (ESP32) nas portarias + hub acadêmico integrado
Proposta de valor	Acesso seguro via QR temporário + grade, faltas e salas centralizados em um só lugar; logs de entrada em tempo real para a gestão
Público primário	Alunos da Faculdade Senac PE
Público secundário	Coordenação acadêmica, gestão administrativa, equipe de segurança
Link Figma	https://www.figma.com/proto/vSTdySE0afZCWpwekHlQ86/Untitled?node-id=0-1

2.2 Equipe e Subgrupos

Hardware IoT	Amanda Cruz, Phellipe Leandro
Frontend	Isabel Vitória, Lucas Eloi
Backend e Banco de Dados	Nikolas Messias, Ronald Paixão, Vinicius Manoel
Documentação e Gestão	Gilberto Quintino

2.3 Lista de Componentes de Hardware

Componente	Função	Especificações Técnicas
ESP32-WROVER-DEV	Controlador principal IoT	Wi-Fi 802.11 b/g/n nativo; GPIO programável; suporte a câmera
Módulo câmera OV2640 (ESP32-CAM)	Captura de imagem para leitura de QR Code	2 MP (até 1600×1200); interface SCCB/I2C; 3,3V; integrada ao módulo ESP32-CAM
Sensor ultrassônico HC-SR04	Deteção de presença/aproximação antes da leitura do QR Code	Alcance: 2 cm – 4 m; precisão: ±3 mm; ângulo: 15°; frequência: 40 kHz; tensão: 5V DC
Servo motor TowerPro MG996R	Acionamento mecânico da catraca/cancela	Torque: 9,4 kgf.cm (4,8V); giro: 180°; tensão: 4,8–7,2V; engrenagens metálicas
Protoboard (breadboard)	Montagem do circuito sem solda	—
Módulo de alimentação MB102	Distribuição de energia na protoboard	Saídas de 3,3V e 5V; compatível com protoboard padrão

Componente	Função	Especificações Técnicas
Fonte de alimentação DC	Alimentação elétrica do sistema	—
Cabo USB/micro-USB	Programação e alimentação do ESP32	Usado para upload de firmware via computador
Jumpers/conectores de ligação	Conexões elétricas entre componentes na protoboard	Macho-macho e macho-fêmea conforme necessário

2.4 User Stories

ID	Prior.	História de Usuário	Critério de Aceitação
US01	Alta	Como aluno, quero gerar QR Code dinâmico no app para validar entrada.	QR gerado após autenticação; válido por 5 minutos
US02	Alta	Como gestor, quero leitura de QR via ESP32 para registro de presença.	Leitura processada em tempo real com envio de sinal ao sistema
US03	Alta	Como aluno, quero ver grade e sala no app logo após validar a entrada.	Sistema cruza ID do aluno com BD acadêmico e exibe local da aula
US04	Média	Como aluno, quero consultar minhas faltas no dashboard do app.	Faltas atualizadas conforme registro de entrada no campus
US05	Média	Como coordenador, quero log de acessos com nome, ID, data, hora e sala.	Log com: Nome, ID, Data, Hora, Sala

2.5 Fluxo Operacional (9 Etapas)

#	Etapas	Descrição
2	Geração do QR Code	O aluno realiza login no aplicativo mobile (React Native)
3	Detecção de presença	O aplicativo solicita ao backend um QR Code temporário e exclusivo (válido por 5 minutos após autenticação)
		O HC-SR04 detecta a aproximação do aluno (distância abaixo do limiar definido) e aciona a câmera
4	Captura pelo hardware	O ESP32-CAM + OV2640 captura a imagem e o ESP32 decodifica o QR Code
5	Envio via MQTT	O ESP32 envia os dados decodificados ao backend via protocolo MQTT (Mosquitto Broker)
6	Validação	O backend (NestJS) valida a autenticidade do QR Code e a identidade do usuário
7	Liberação do acesso	O acesso é liberado: servo MG996R aciona a catraca/cancela
8	Registro do evento	O backend registra no PostgreSQL: data, hora, identidade e resultado do acesso
9	Retorno ao usuário	As informações acadêmicas atualizadas (horário e sala) são disponibilizadas no app e no painel de gestão

2.6 Stack Tecnológico

Tecnologia	Camada	Papel no Sistema
React Native	Mobile	App do aluno — login, geração de QR, consulta de dados (Android + iOS)
Node.js + NestJS	Backend	Monolito modular (Controllers, Services, Modules); regras de negócio
PostgreSQL	Banco de Dados	Dados acadêmicos + logs de acesso; banco relacional robusto
MQTT / Mosquitto	Comunicação IoT	Eventos em tempo real entre ESP32 e backend; baixa latência
JWT	Segurança	Autenticação segura no app; controle de acesso e proteção de rotas
HTTPS	Segurança IoT	
ESP32	/ Portaria	Comunicação criptografada entre app e backend Lê QR Code, detecta entrada, aciona cancela; Wi-Fi integrado

2.7 Requisitos do Sistema

Requisitos Funcionais

RF01	Gerar QR Code dinâmico e temporário (válido por 5 minutos após autenticação)
RF02	Validar QR Code através do dispositivo IoT (ESP32)
RF03	Registrar entradas dos usuários (data, hora, identidade, local)
RF04	Permitir consulta de horários e informações acadêmicas no app
RF05	Disponibilizar histórico de acessos para a gestão
RF06	Acionar o mecanismo de liberação física (servo motor / catraca)

Requisitos Não-Funcionais

RNF01	Baixa latência na validação (tempo real via MQTT)
RNF02	Disponibilidade contínua do sistema (alta disponibilidade)
RNF03	Segurança na comunicação e armazenamento (HTTPS, JWT, criptografia)
RNF04	Escalabilidade para múltiplos usuários simultâneos
RNF05	Proteção das credenciais (sem exposição de chaves no código)

2.8 Cibersegurança e LGPD

Medida	Descrição
QR Code temporário	Geração apenas após autenticação; validade de 5 minutos; validação única — impede fraudes por print ou empréstimo de credenciais
HTTPS	Comunicação criptografada entre app e backend; impede interceptação de dados sensíveis
JWT	Autenticação segura no app; controle de acesso por perfil; proteção das rotas da API

Medida	Descrição
Controle de acesso	Credenciais protegidas; arquivos de configuração isolados (.env); chaves nunca expostas no código-fonte
Minimização de dados	Logs registram apenas o necessário: quem acessou, quando e onde (portaria)
Proteção de credenciais	Arquivo .env.example com chaves fictícias; real no .gitignore; uso ético declarado no README

2.9 Mapeamento das Unidades Curriculares (UCs)

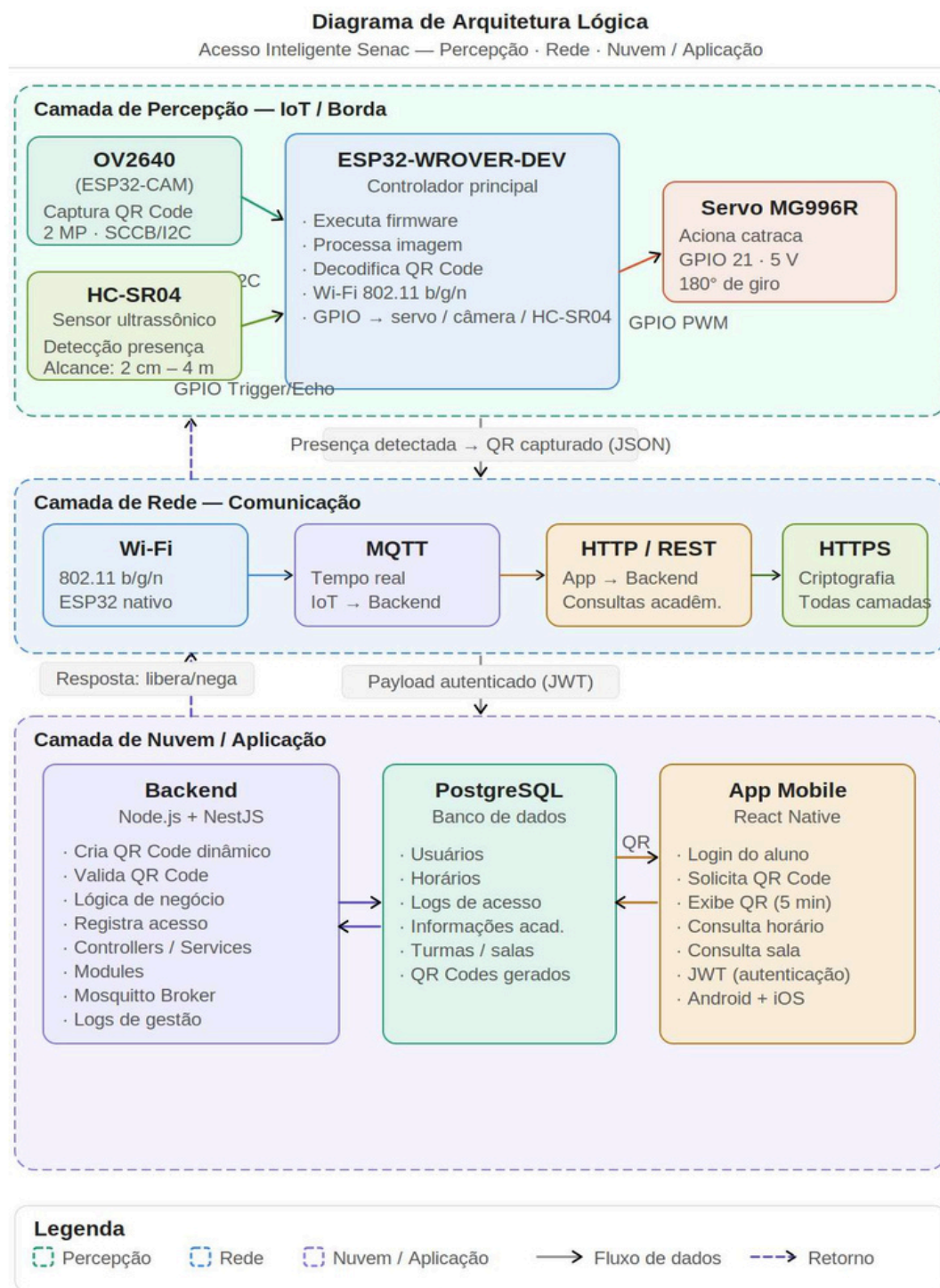
Unidade Curricular	Evidência no Projeto
IoT	ESP32-WROVER-DEV, câmera OV2640 (ESP32-CAM), sensor ultrassônico HC-SR04, servo motor MG996R; firmware embarcado com leitura de sensores e acionamento de atuadores
Cloud Computing	Broker MQTT (Mosquitto), backend em nuvem (NestJS), banco de dados PostgreSQL; dashboards com dados históricos
Engenharia de Software	APIs REST (HTTP), comunicação MQTT, arquitetura modular NestJS (Controllers, Services, Modules), PostgreSQL
Segurança da Informação	JWT para autenticação, HTTPS na comunicação, QR Code temporário, proteção de credenciais, minimização de dados (LGPD)
Comportamento do Consumidor	Mapeamento da dor do aluno (informações dispersas, acesso fragmentado); persona e jornada do usuário; público-alvo primário e secundário
Inglês	README técnico em inglês; documentação e comentários de código; seção de abstract do projeto
Projeto Integrador	Integração completa do ecossistema: mobile + IoT + backend + banco de dados + comunicação em tempo real

2.10 Marcas Formativas Senac

Marca Formativa	Como é evidenciada no projeto
Domínio Técnico-Científico	Integração entre hardware (ESP32), software (NestJS), rede (MQTT/HTTPS) e banco de dados (PostgreSQL); rigor no firmware e modelagem do payload JSON
Resolução de Problemas com Autonomia Digital	Integração de APIs, tratamento de erros de conexão IoT-backend, depuração autônoma de hardware e software
Visão Crítica, Ética e Segurança	Aplicação de LGPD, QR Code temporário, autenticação segura, proteção de credenciais, travas fail-safe
Comunicação e Colaboração	Divisão de tarefas por subequipes, GitHub com README profissional, metodologia Scrum/Kanban
Atitude Empreendedora e Inovadora	Digitalização e automação do controle de acesso acadêmico; proposta de valor clara; centralização de informações dispersas

4. Diagrama de Arquitetura Lógica

O diagrama abaixo representa a arquitetura lógica do sistema em três camadas: Percepção (IoT/Borda), Rede (Comunicação) e Nuvem/Aplicação. Evidencia o fluxo de dados desde a leitura do QR Code pelo ESP32 até o registro no banco de dados e o retorno da resposta ao dispositivo físico.

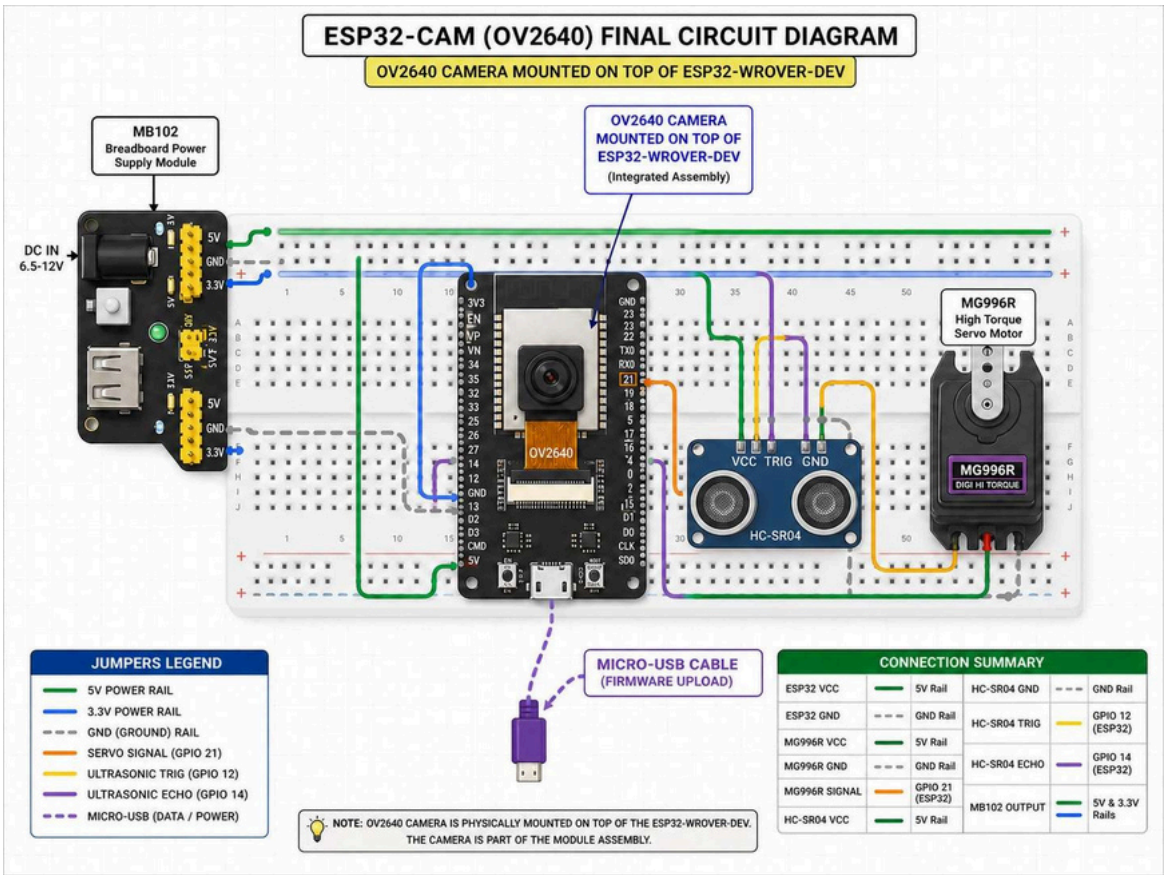


Acesso Inteligente Senac · 07/06/2026 · rev. 3 — HC-SR04 reinserido

5. Esquema Elétrico de Ligação (P01)

O diagrama abaixo apresenta o esquema elétrico do circuito montado na protoboard, elaborado com os componentes confirmados do projeto. A câmera OV2640 está fisicamente montada sobre o ESP32-WROVER-DEV (montagem integrada). As conexões seguem a pinagem definida pelo time de hardware:

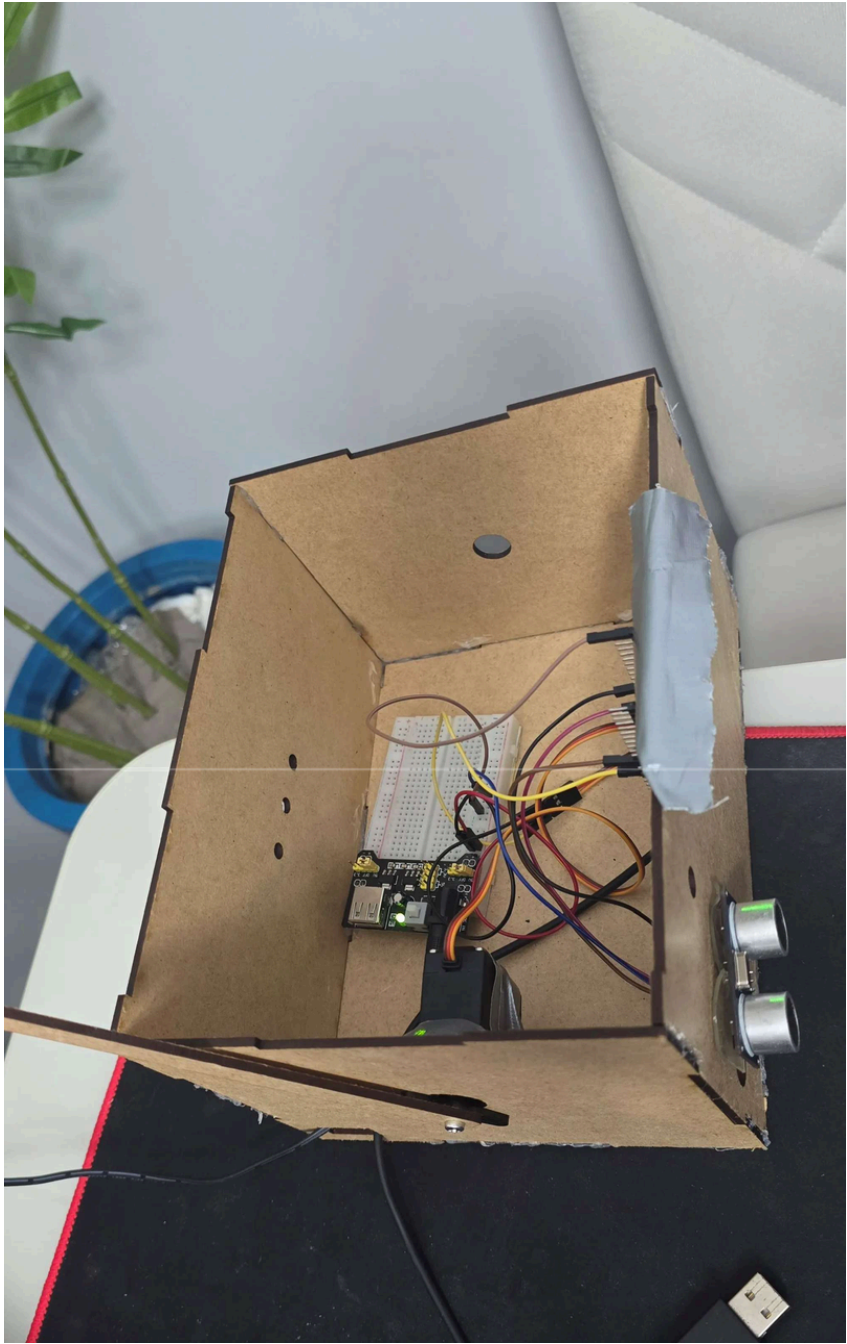
Componente	Pino / Rail	Componente	Pino / Rail
ESP32 VCC	5V Rail	HC-SR04 GND	GND Rail
ESP32 GND	GND Rail	HC-SR04 TRIG	GPIO 12 (ESP32)
MG996R VCC	5V Rail	HC-SR04 ECHO	GPIO 14 (ESP32)
MG996R GND	GND Rail	HC-SR04 VCC	5V Rail
MG996R SIGNAL	GPIO 21 (ESP32)	MB102 OUTPUT	5V e 3.3V Rails



Nota: a câmera OV2640 é parte do módulo ESP32-CAM, montada fisicamente sobre o ESP32-WROVER-DEV.

6. Registro Fotográfico do Protótipo Físico (P01)

A imagem abaixo registra o protótipo físico do sistema Acesso Inteligente Senac em construção. É possível identificar a estrutura da caixa em MDF cortada a laser, a protoboard com os componentes conectados (MB102, ESP32-WROVER-DEV, jumpers e fiação), o servo motor MG996R e o sensor ultrassônico HC-SR04 fixado na lateral direita da caixa.



(Amanda Cruz e Phellipe Leandro) — 07/06/2026.

Elemento visível	Descrição
Caixa em MDF	Estrutura física do protótipo cortada a laser; serve de gabinete para os componentes eletrônicos
Protoboard	Breadboard com os componentes conectados via jumpers; visível no interior da caixa

Elemento visível	Descrição
MB102	Módulo de alimentação da protoboard; identificável pelo LED verde de energia
Jumpers / fiação	Cabos coloridos conectando os componentes (amarelo, vermelho, azul, laranja, preto)
Servo MG996R	Motor de acionamento da catraca; posicionado na parte frontal/inferior da caixa
HC-SR04	Sensor ultrassônico de presença; fixado na lateral direita com os transdutores voltados para fora
Cabo USB	Visível na parte inferior; utilizado para upload de firmware e alimentação

7. Serial Monitor — Evidência de Funcionamento (P03)

Os prints abaixo foram capturados do Serial Monitor do Arduino IDE durante execução real do firmware no ESP32-WROVER-DEV. Evidenciam dois ciclos completos: um de acesso liberado (QR Code válido) e um de acesso negado (QR Code inválido ou ilegível).

7.1 Print completo do Serial Monitor

```

Output Serial Monitor X
Not connected. Select a board and a port to connect automatically.
20:39:44.882 -> Distancia: 25.0 cm
20:39:45.183 -> Distancia: 24.2 cm
20:39:45.492 -> Distancia: 25.5 cm
20:39:45.764 -> Distancia: 26.3 cm
20:39:46.068 -> Distancia: 26.2 cm
20:39:46.390 -> Distancia: 26.7 cm
20:39:46.717 -> Distancia: 27.4 cm
20:39:47.481 -> Distancia: 21.2 cm
20:39:47.790 -> Distancia: 8.4 cm
20:39:47.790 -> Pessoa detectada! Capturando foto...
20:39:48.290 -> Foto: 33781 bytes
20:39:50.238 -> HTTP 200 | Resposta: {"valid":true,"action":"OPEN","student":{"id":"f770d47a-3e15-4cad-9cb8-07ade7c53490","name
20:39:50.238 -> Acesso liberado! Abrindo cancela...
20:39:55.255 -> Cancela fechada.
20:39:58.522 -> Distancia: 38.2 cm
20:39:58.867 -> Distancia: 41.9 cm
20:39:59.178 -> Distancia: 40.2 cm
20:39:59.644 -> Distancia: 21.8 cm
20:39:59.903 -> Distancia: 15.4 cm
20:39:59.903 -> Pessoa detectada! Capturando foto...
20:40:00.403 -> Foto: 26788 bytes
20:40:04.479 -> HTTP 400 | Resposta: {"action":"DENY","reason":"QR Code não encontrado na imagem. Verifique iluminação e enquadr
20:40:04.479 -> Acesso negado.
  
```

7.2 Zoom — ciclo de acesso liberado

```

20:39:47.790 -> Distancia: 8.4 cm
20:39:47.790 -> Pessoa detectada! Capturando foto...
20:39:48.290 -> Foto: 33781 bytes
20:39:50.238 -> HTTP 200 | Resposta: {"valid":true,"action":"OPEN","student":{"id":"f770d47a-3e15-4cad-9cb8-07ade7c53490","name
20:39:50.238 -> Acesso liberado! Abrindo cancela...
20:39:55.255 -> Cancela fechada.
  
```

7.3 Análise linha a linha

Timestamp	Mensagem no Serial Monitor	O que evidencia
20:39:44-47	Distancia: 25.0 cm ... 8.4 cm	HC-SR04 lendo continuamente; distância cai de ~26 cm para 8,4 cm (presença detectada)
20:39:47.790	Pessoa detectada! Capturando foto...	Limiar atingido: firmware aciona OV2640 para captura de imagem
20:39:48.290	Foto: 33781 bytes	Imagem capturada com sucesso — 33.781 bytes enviados ao backend
20:39:50.238	HTTP 200 Resposta: {"valid":true,"action":"OPEN",...}	Backend validou o QR Code; retorna ação OPEN e ID do estudante
20:39:50.238	Acesso liberado! Abrindo cancela...	Firmware aciona servo MG996R — cancela abre
20:39:55.255	Cancela fechada.	Servo retorna à posição 0° após delay — ciclo concluído (~7,5 s)
20:39:58-59	Distancia: 38.2 cm ... 15.4 cm	Novo ciclo iniciado — nova aproximação detectada
20:39:59.903	Pessoa detectada! Capturando foto...	Segunda detecção: câmera acionada novamente

Timestamp	Mensagem no Serial Monitor	O que evidencia
20: 40: 0 0.40 3	Foto: 26788 bytes	Segunda imagem capturada — tamanho menor (26.788 bytes)
20: 40: 0 4.47 9	HTTP 400 Resposta: {"action": "DENY", "reason": "QR Code não encontrado..."}	QR Code inválido ou ilegível — backend nega acesso
20: 40: 0 4.47 9	Acesso negado.	Firmware recebe DENY — cancela permanece fechada

✓ Ciclo de acesso liberado

Duração: ~7,5 segundos (20:39:47 → 20:39:55)
Detecção → Captura → Validação → Abertura → Fechamento

✗ Ciclo de acesso negado

HTTP 400: QR Code não encontrado na imagem
Cancela permanece fechada — fail-safe ativo

8. Payload JSON e Endpoint da API (P04 / P05)

Os payloads abaixo foram extraídos diretamente do Serial Monitor durante execução real do sistema. Representam as respostas do backend ao ESP32 nos dois cenários possíveis: acesso liberado e acesso negado.

8.1 Endpoint identificado

Campo	Valor
Método	HTTP POST
Endpoint	/api/acesso (confirmado via HTTP 200/400 no Serial Monitor)
Protocolo	HTTP/REST (camada de validação); MQTT previsto para eventos em tempo real
Repositório Backend	https://github.com/ViniciusMLAr a uj o/Pr oj etoP IBac k /tr ee/fe at/r on i-foundations
Repositório Frontend	https://github.com/isabel-vit309/Front-Smart-Campus

8.2 Payload de resposta — Acesso Liberado (HTTP 200)

```
{
  "valid": true,
  "action": "OPEN",
  "student": {
    "id": "f770d47a-3e15-4cad-9cb8-07ade7c53490",
    "name": "[nome do estudante]"
  }
}
```

8.3 Payload de resposta — Acesso Negado (HTTP 400)

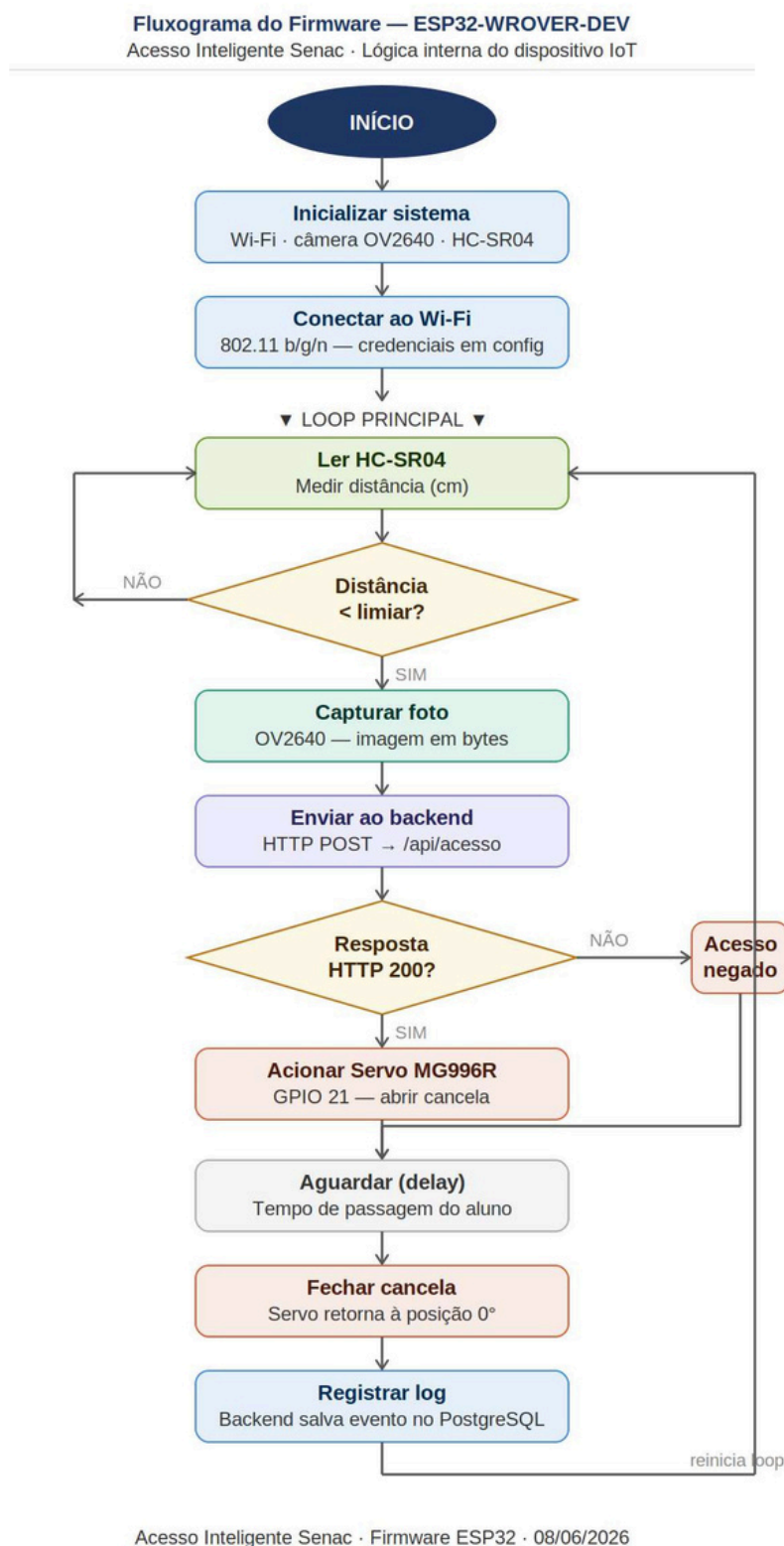
```
{
  "action": "DENY",
  "reason": "QR Code não encontrado na imagem. Verifique iluminação e enquadramento."
}
```

i Observação sobre o protocolo

O Serial Monitor evidencia comunicação via HTTP POST durante a validação do QR Code. A arquitetura prevê MQTT para eventos em tempo real entre ESP32 e backend (Mosquitto Broker). Ambos os protocolos são camadas distintas e complementares: HTTP para requisições síncronas de validação, MQTT para publicação de eventos assíncronos de acesso — conforme documentado na Seção 4 (Diagrama de Arquitetura).

9. Fluxograma do Firmware — ESP32 (P08)

O fluxograma abaixo representa a lógica interna do firmware embarcado no ESP32-WROVER-DEV, mapeada a partir do comportamento observado no Serial Monitor e da arquitetura definida pela equipe. Evidencia o fluxo de decisão desde a inicialização até o acionamento da cancela e o reinício do loop.



9.1 Descrição das etapas

#	Etapa	Descrição
1	Inicializar sistema	Configura Wi-Fi, câmera OV2640 e HC-SR04; carrega credenciais do arquivo de configuração
2	Conectar ao Wi-Fi	Estabelece conexão 802.11 b/g/n com o ponto de acesso configurado
3	Ler HC-SR04	Mede distância em centímetros continuamente no loop principal
4	Distância < limiar?	Se a distância medida estiver abaixo do limiar (presença detectada), avança; caso contrário, reinicia a leitura
5	Capturar foto	OV2640 captura imagem do QR Code apresentado pelo aluno
6	Enviar ao backend	HTTP POST com a imagem para o endpoint /api/acesso
7	Resposta HTTP 200?	Se o backend retornar HTTP 200 (válido), avança para abertura; HTTP 400 (negado) volta ao loop
8	Acionar Servo MG996R	GPIO 21 envia sinal PWM ao servo — cancela abre (posição 90°) Tempo definido para o aluno passar pela cancela
9	Aguardar (delay)	Servo retorna à posição 0° — cancela fecha
10	Fechar cancela	Backend persiste o evento no PostgreSQL (data, hora, ID, resultado)
11	Registrar log	

7. Demonstração do Primeiro Sinal — Serial Monitor (P02)

O print abaixo registra a primeira execução funcional completa do firmware no ESP32-WROVER-DEV, capturada via Serial Monitor do Arduino IDE. O log evidencia o ciclo completo de funcionamento do sistema: detecção de presença, captura de imagem, comunicação com o backend e acionamento físico da cancela.

```
20:39:47.790 -> Distancia: 8.4 cm
20:39:47.790 -> Pessoa detectada! Capturando foto...
20:39:48.290 -> Foto: 33781 bytes
20:39:50.238 -> HTTP 200 | Resposta: {"valid":true,"action":"OPEN","student":{"id":"f770d47a-3e15-4cad-9cb8-07ade7c53490","name":
20:39:50.238 -> Acesso liberado! Abrindo cancela...
20:39:55.255 -> Cancela fechada.
```

7.1 Análise do Log

#	Linha do Serial Monitor	O que evidencia
1	20:39:47.790 → Distancia: 8.4 cm	HC-SR04 ativo e lendo distância em tempo real — presença detectada a 8,4 cm (abaixo do limiar)
2	20:39:47.790 → Pessoa detectada! Capturando foto...	Lógica de decisão do firmware funcionando: limiar atingido aciona a câmera OV2640
3	20:39:48.290 → Foto: 33781 bytes	Câmera OV2640 capturou imagem com sucesso — 33.781 bytes de dados de imagem gerados
4	20:39:50.238 → HTTP 200 Resposta: {"valid":true,...}	Comunicação com o backend bem-sucedida — QR Code validado; resposta retorna action: OPEN e ID do estudante
5	20:39:50.238 → Acesso liberado! Abrindo cancela...	Firmware processa resposta do backend e aciona o servo MG996R para abrir a cancela
6	20:39:55.255 → Cancela fechada.	Servo retorna à posição fechada após o tempo definido — ciclo completo concluído com sucesso

✓ Ciclo completo funcional confirmado

O log demonstra que todos os componentes do sistema estão integrados e operacionais: HC-SR04 (detecção) → OV2640 (captura) → Backend via HTTP/MQTT (validação) → Servo MG996R (acionamento). O tempo total do ciclo foi de aproximadamente 7,5 segundos (20:39:47 → 20:39:55).

7.2 Desmonstração do Código usado na implementação

O sistema consiste em um dispositivo IoT baseado no microcontrolador ESP32, projetado para o gerenciamento automatizado de acesso. O firmware foi desenvolvido para ser não bloqueante, garantindo estabilidade operacional através do uso de temporizadores baseados em `millis()` em vez de pausas de execução (`delay()`). O fluxo lógico garante segurança mecânica e eficiência, integrando sensores ultrassônicos, atuadores servo e sinalização LED.

2. Ferramentas e Tecnologias

Para a realização deste projeto, foram utilizadas as seguintes ferramentas e tecnologias:

- Hardware (Simulação): Wokwi Simulator para prototipagem e testes de componentes físicos (ESP32, HC-SR04, Servo Motor e LEDs).
- Linguagem de Programação: C++ (Framework Arduino) para o desenvolvimento do firmware.
- Gestão de Documentação e Lógica:
 - IA (Colaborador): Gemini (Google) para arquitetura de software, correção de código e estruturação de fluxogramas.
 - Mermaid.js: Utilizado para a criação da representação visual do fluxo de dados e estados do firmware.

```
#include <WiFi.h>           // Biblioteca para conexão Wi-Fi do ESP32
#include <WiFiMulti.h>       // Biblioteca para gerenciar múltiplas redes Wi-Fi
#include <HTTPClient.h>      // Biblioteca para realizar requisições HTTP
#include <ESP32Servo.h>      // Biblioteca para controlar o Servo Motor

// --- DEFINIÇÃO DE PINOS ---
const int PIN_TRIG = 12;    // Pino de disparo do sensor ultrassônico
const int PIN_ECHO = 14;    // Pino de recepção do sensor ultrassônico
const int PIN_SERVO = 13;   // Pino de sinal do servo motor
const int LED_VM = 2;       // Pino do LED Vermelho (Bloqueado)
const int LED_VD = 5;       // Pino do LED Verde (Liberado)

// --- CONFIGURAÇÕES DE VARIÁVEIS ---
const float LIMAR_DISTANCIA = 400.0; // Distância máxima de detecção (em cm)
unsigned long tempoInicioAcao = 0;    // Armazena o tempo em que a cancela abriu
unsigned long tempoUltimoFechamento = 0; // Armazena o tempo em que a cancela fechou
bool cancelaAberta = false;          // Variável de controle do estado da cancela
```

```
// Tempo de espera (3 segundos) para evitar detecções seguidas (Debounce)
const unsigned long PAUSA_ENTRE_CICLOS = 3000;

Servo cancelaServo;        // Cria o objeto para controlar o servo
WiFiMulti wifiMulti;       // Cria o objeto para gerenciar as conexões Wi-Fi

void setup() {
  Serial.begin(115200);     // Inicia a comunicação serial para debug
  pinMode(PIN_TRIG, OUTPUT); // Define o pino TRIG como saída
  pinMode(PIN_ECHO, INPUT); // Define o pino ECHO como entrada
  pinMode(LED_VM, OUTPUT);  // Define o LED Vermelho como saída
  pinMode(LED_VD, OUTPUT);  // Define o LED Verde como saída

  cancelaServo.attach(PIN_SERVO, 500, 2400); // Conecta o servo ao pino 13
  cancelaServo.write(0);    // Coloca o servo na posição inicial (fechado)

  digitalWrite(LED_VM, HIGH); // Liga o LED Vermelho no início
  digitalWrite(LED_VD, LOW);  // Garante o LED Verde desligado no início
```

```
// Adiciona as redes Wi-Fi disponíveis
wifiMulti.addAP("Wokwi-GUEST", "");
wifiMulti.addAP("Wi-Fi-Senac", "Senac2026");
}

void loop() {
  wifiMulti.run(); // Mantém a conexão Wi-Fi ativa
  unsigned long tempoAtual = millis(); // Captura o tempo atual do sistema

  // --- ROTINA DE LEITURA DO SENSOR ---
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH); // Dispara o pulso do sensor
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
```

```
// Calcula a distância baseada no tempo de resposta do eco
long duracao = pulseIn(PIN_ECHO, HIGH, 20000);
float distancia = (duracao == 0) ? 999.0 : duracao * 0.034 / 2;

// --- LÓGICA DE ABERTURA ---
// Verifica se: cancela está fechada, objeto detectado E tempo de pausa decorrido
if (!cancelaAberta && distancia > 0 && distancia <= LIMAR_DISTANCIA && (tempoAtual - tempoInicioAcao >= PAUSA_ENTRE_CICLOS)) {
  cancelaAberta = true; // Define que a cancela agora está aberta
  tempoInicioAcao = tempoAtual; // Grava o momento da abertura

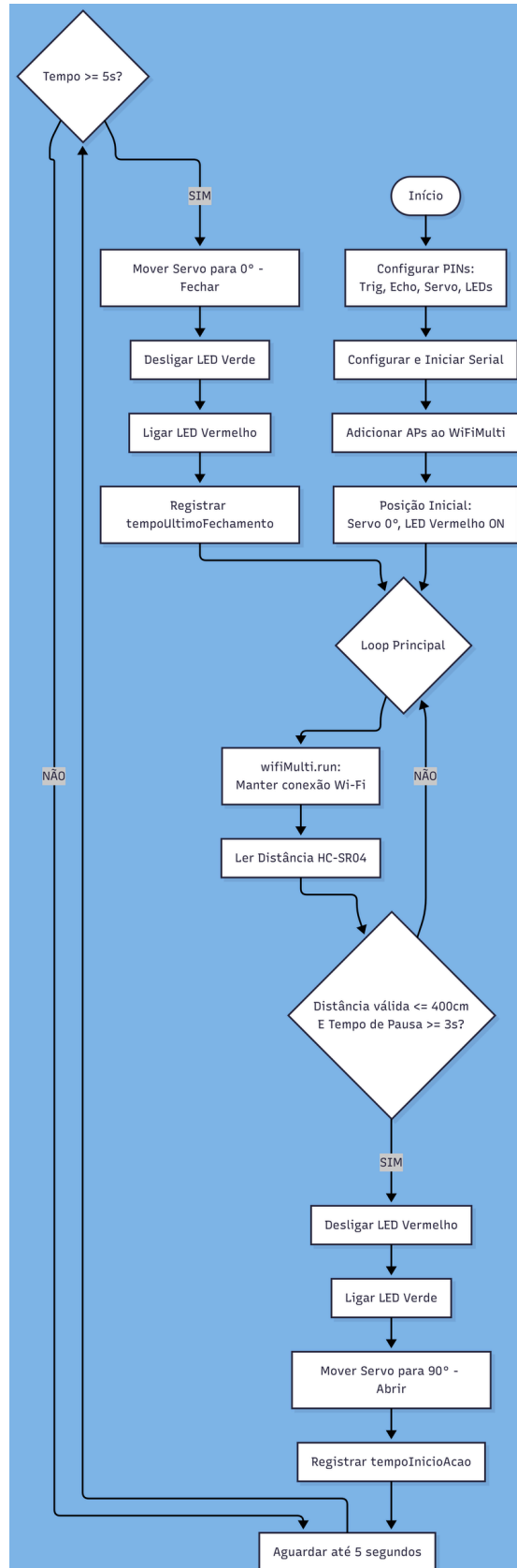
  digitalWrite(LED_VM, LOW); // Apaga o LED Vermelho
  digitalWrite(LED_VD, HIGH); // Acende o LED Verde
  cancelaServo.write(90);    // Move o servo para abrir a cancela
  Serial.println("Acesso liberado! Aberta por 5 segundos.");
}
```

```
// --- LÓGICA DE FECHAMENTO ---
// Verifica se a cancela está aberta E se passaram 5 segundos (5000ms)
if (cancelaAberta && (tempoAtual - tempoInicioAcao >= 5000)) {
  cancelaAberta = false; // Define que a cancela agora está fechada
  tempoUltimoFechamento = tempoAtual; // Grava o momento do fechamento

  digitalWrite(LED_VD, LOW); // Apaga o LED Verde
  digitalWrite(LED_VM, HIGH); // Acende o LED Vermelho
  cancelaServo.write(0);    // Move o servo para fechar a cancela
  Serial.println("Cancela fechada. Aguardando...");
}
```

11. Fluxograma de Processos

- A lógica do sistema foi documentada através de fluxogramas técnicos (Gerados via Mermaid), que descrevem:
- A inicialização dos periféricos.
- A verificação contínua da distância pelo sensor HC-SR04.
- As condições de decisão para abertura e fechamento da cancela.
- O ciclo de pausa de segurança.



10. Tópicos Desenvolvidos

- Arquitetura de Firmware: Criação de uma máquina de estados finitos (FSM) que transita entre os estados de "Bloqueado" (LED Vermelho) e "Liberado" (LED Verde).
- Otimização de Código: Implementação de técnicas de programação assíncrona, garantindo que o sistema monitore a rede Wi-Fi (WiFiMulti) e o sensor simultaneamente.
- Segurança Operacional: Implementação de uma trava de segurança (debounce de 3 segundos) entre ciclos de abertura para evitar desgaste mecânico do servo motor e detecções erráticas.
- Gerenciamento de Tempo: Configuração precisa de 5 segundos de abertura, com retorno automático ao estado de repouso.

